

Operational and Deployment Guide

Installation, Training Pipeline, Backtesting,
and Live Production Deployment of the GBT+RL Trading System

Kripta Studios Quantitative Research Division

March 2026

Contents

1	Prerequisites and System Requirements	2
1.1	Hardware Specifications	2
1.2	Software Dependencies	2
1.3	ThetaData Terminal	2
2	Complete Training Pipeline	3
2.1	Stage 1: Historical Data Collection	3
2.2	Stage 2: GBT Walk-Forward Training	4
2.3	Stage 3: Episode Index Generation	5
2.4	Stage 4: RL Preprocessing	5
2.5	Stage 5: PPO Training	6
3	Backtesting	6
3.1	GBT-Only Backtest	6
3.2	GBT+RL Backtest	7
3.3	Position Sizing and Risk Management	7
4	Live Trading Deployment	7
4.1	System Architecture	8
4.2	Environment Configuration	8
4.3	Step-by-Step Startup Sequence	8
4.3.1	Step 1: Start ThetaTerminal Proxy	8
4.3.2	Step 2: Launch the Real-Time Data Feed	9
4.3.3	Step 3: Launch the Trading Bot	10
4.4	Dual-Ticker Processing Pipeline	10
4.5	Market Hours and Safety	11
4.6	Discord Alert System	11
4.6.1	GBT Directional Signals — [GBM] Tag	11
4.6.2	RL Trade Execution — [RL] Tag	12

4.6.3	Bot Status Messages	12
4.7	Expiration Resolution Logic	12
4.8	Historical Data Requirements	13
5	Visualization and Diagnostics	13
5.1	Trade Chart	13
5.2	Feature Importance	13
5.3	Model Registry Inspection	13
6	Operational Checklist	14

1 Prerequisites and System Requirements

1.1 Hardware Specifications

Component	Minimum	Recommended / Notes
CPU Cores	8 logical	24+ for full parallel preprocessing ($W = 24$ workers)
RAM	32 GB	64 GB for RL training with <code>max_days_in_ram = 120</code>
Storage	NVMe SSD	≥ 200 GB free; options archive ≈ 80 GB compressed
GPU	None required	Training is CPU-bound (LightGBM) and small-MLP (PPO)
Network	Broadband	ThetaTerminal requires stable connection to ThetaData servers
OS	Windows 10+	PowerShell 7+ required for <code>run_pipeline.ps1</code>

Table 1: Hardware and operating environment requirements.

1.2 Software Dependencies

The system is implemented in Python 3.10+ and managed via a standard `requirements.txt`. Key dependencies include:

Package	Version	Role
<code>lightgbm</code>	≥ 4.0	GBT directional classifier
<code>torch</code>	≥ 2.0	PPO agent (small CPU MLP, 187K params)
<code>pyarrow</code>	≥ 12.0	Parquet I/O with snappy compression
<code>pandas</code>	≥ 2.0	Data manipulation
<code>numpy</code>	≥ 1.24	Numerical operations
<code>httpx</code>	≥ 0.25	Async ThetaData API client
<code>joblib</code>	≥ 1.3	GBT model serialization
<code>tqdm</code>	any	Progress bars

Table 2: Core Python dependencies.

1.3 ThetaData Terminal

All historical and real-time data is sourced from ThetaData via the ThetaTerminal local proxy. Before running any data ingestion command, the terminal must be active:

```
java -jar ThetaTerminalv3.jar
```

The terminal proxies all API requests to `http://127.0.0.1:25503/v3`. An active ThetaData subscription with historical options access is required. The system requests OHLC, Greeks, and open interest for SPXW, SPX, SPY, QQQ, VIX, and TLT.

2 Complete Training Pipeline

The full pipeline to train and deploy a production GBT+RL system is a sequential five-stage procedure, illustrated in Figure 1.

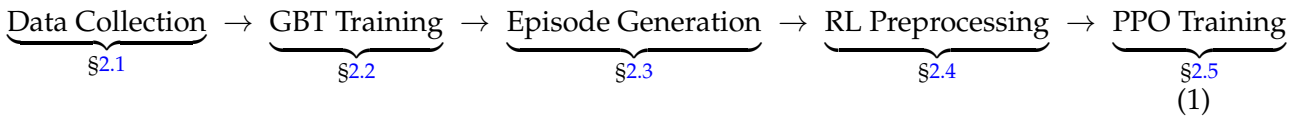


Figure 1: Five-stage training pipeline. Each stage must complete before the next begins.

The unified PowerShell orchestrator executes Stages 1–3 automatically:

```
.\neural\run_pipeline.ps1 -Update true
```

This script handles market holiday detection (via `last_trading_day_of_week`) to resolve non-standard expiration dislocations in quarterly and holiday-shortened weeks.

2.1 Stage 1: Historical Data Collection

Download raw options OHLC + Greeks from ThetaData:

```
# Options chains (all strikes, OHLC + IV + Greeks)
python services/options_downloader.py \
  --symbols SPXW SPX SPY QQQ VIX TLT \
  --start 2024-01-01 \
  --end 2026-02-28

# Derive underlying OHLC from options (spot price proxy)
python services/underlying_downloader.py \
  --symbols SPXW SPX SPY QQQ VIX TLT \
  --start 2024-01-01 \
  --end 2026-02-28

# Fix zero-gaps at 09:30 open (critical pre-processing step)
python services/data_corrector.py \
  --data-dir D:/ThetaData/data_underlying_derived \
  --symbols SPXW SPY QQQ VIX TLT
```

Then build the flat training design matrix $\mathbf{X} \in \mathbb{R}^{N \times 163}$:

```
python neural/collect_training_data_parquet.py \
--start 20240101 \
--end 20260220 \
--workers 26 \
--output training_data_derived.parquet
```

2.2 Stage 2: GBT Walk-Forward Training

Train the LightGBM directional ensemble using anchored walk-forward cross-validation:

```
python neural/train_walkforward.py \
--data training_data/training_data_derived.parquet \
--train-months 6 \
--test-months 1 \
--ensemble 3 \
--top-n-windows 10 \
--model_path models/trading_hybrid_wf.joblib \
--norm_path models/hybrid_normalizer_wf.npz
```

Argument	Default	Description
-train-months	6	Training window size in months
-test-months	1	Validation window size in months
-ensemble	3	Number of LightGBM models per window (different seeds)
-top-n-windows	10	Top-N walk-forward windows by rank_score to include
-min-window	0	Ignore windows before this index (recency filtering)

Table 3: Key arguments for `train_walkforward.py`.

The training procedure:

1. Generates walk-forward splits (train window $\rightarrow$ test window, step 1 month).
2. For each window, trains N_{ensemble} LightGBM classifiers with seeds $\{42, 43, \dots, 42 + N_{\text{ensemble}} - 1\}$.
3. Rejects models with collapse ($> 90\%$ one-class predictions), fewer than 20 test trades, or profit factor < 0.30 .
4. Registers accepted windows and ranks by a recency-weighted score:

$$\text{rank_score}_w = \text{PF}_w \cdot \left(\frac{w}{W_{\text{total}}} \right)^2 \quad (2)$$

5. Flattens all GBTModels from the top- N windows into a single GBTEnsemble and serializes via joblib.

Outputs: models/trading_hybrid_wf.joblib, models/hybrid_normalizer_wf.npz, models/window_reg

2.3 Stage 3: Episode Index Generation

Run GBT inference over the training data to create the episode index for RL training:

```
import torch, numpy as np, pandas as pd
from hybrid_model import FEATURE_COLUMNS, FeatureNormalizer
from gbt_model import load_gbt_ensemble

ensemble, normalizer = load_gbt_ensemble(
    'models/trading_hybrid_wf.joblib',
    'models/hybrid_normalizer_wf.npz'
)

df = pd.read_parquet('training_data/training_data_derived.parquet')
feats = normalizer.transform(df[FEATURE_COLUMNS].values.astype('float32'))

# Batch inference
probs_list = []
for i in range(0, len(feats), 4096):
    batch = feats[i:i+4096]
    logits, _ = ensemble(batch)
    probs_list.append(logits.numpy())
probs = np.concatenate(probs_list)

dm = {0: 'SHORT', 1: 'HOLD', 2: 'LONG'}
df['mlp_direction'] = [dm[p] for p in np.argmax(probs, 1)]
df['mlp_confidence'] = probs.max(1)

mask = df['mlp_direction'].isin(['LONG', 'SHORT']) \
    & (df['mlp_confidence'] >= 0.55)
ep = df[mask].copy().reset_index(drop=True)
ep['episode_id'] = range(len(ep))
ep.to_parquet('rl_data/episode_index.parquet', index=False)
```

2.4 Stage 4: RL Preprocessing

Build per-day options cache shards for the RL environment:

```
cd neural
python run_preprocess.py \
    --training-data ../training_data/training_data_derived.parquet \
    --options-dir D:/ThetaData/data_options \
    --output ../rl_data/rl_options_cache_chunks \
    --num-workers 24
```

This generates one YYYYMMDD.pkl shard per trading day in `rl_data/rl_options_cache_chunks/`, keyed by "YYYYMMDD_HH:MM" timestamps. Preprocessing time scales as $\mathcal{O}(M/W)$ where M is the number of trading days and $W = 24$ workers.

2.5 Stage 5: PPO Training

Train the RL agent (187,348-parameter PPO network):

```
cd neural
python -m rl.training \
  --episode-index ../rl_data/episode_index.parquet \
  --options-cache ../rl_data/rl_options_cache_chunks \
  --save-dir ../rl_models \
  --total-updates 200 \
  --max-days-in-ram 120
```

Argument	Default	Description
-total-updates	200	Number of PPO gradient update steps
-episodes-per-update	64	Rollout episodes collected per PPO step
-max-days-in-ram	120	LRU cache size: max daily shards in RAM

Table 4: Key arguments for `rl/training.py`.

Output: `rl_models/best_rl_agent.pt` (selected by maximum evaluation profit factor).

3 Backtesting

3.1 GBT-Only Backtest

Simulates directional trading on spot (synthetic futures) using fixed targets and stops:

```
python neural/backtest_hybrid_parquet.py \
  --data training_data/training_data_backtest.parquet \
  --model models/trading_hybrid_wf.joblib \
  --normalizer models/hybrid_normalizer_wf.npz \
  --model-size small \
  --ensemble \
  --threshold 0.55 \
  --cooldown 30 \
  --target_long 0.010 \
  --target_short 0.005 \
  --stop 0.003
```

Argument	Default	Description
-threshold	0.70	Minimum GBT confidence to take a trade
-target_long	0.010	LONG profit target (1% of spot)
-target_short	0.005	SHORT profit target (0.5% of spot)
-stop	0.003	Stop loss (0.3% of spot)
-cooldown	30	Minimum minutes between trades per ticker
-max-time	120	Maximum bars to hold
-uncertainty	45.0	Maximum Bayesian sigma (minutes)

Table 5: Key arguments for `backtest_hybrid_parquet.py`.

3.2 GBT+RL Backtest

Side-by-side comparison of GBT-only spot simulation versus GBT+RL real-options execution:

```
python neural/backtest_rl.py \
  --data training_data/training_data_backtest.parquet \
  --model models/trading_hybrid_wf.joblib \
  --normalizer models/hybrid_normalizer_wf.npz \
  --rl-model rl_models/best_rl_agent.pt \
  --model-size small \
  --ensemble \
  --threshold 0.55 \
  --cooldown 30
```

3.3 Position Sizing and Risk Management

Position sizing scales with GBT confidence:

$$\text{Risk\%} = 10\% + \frac{c_t - 0.55}{0.45} \times 40\%, \quad c_t \in [0.55, 1.00] \quad (3)$$

This maps minimum confidence (0.55) to 10% risk per trade and maximum confidence (1.00) to 50% risk per trade.

After 3 consecutive losses, the system enters drawdown mode:

$$n_{\text{contracts}} \leftarrow \max(1, \lfloor n_{\text{contracts}}/2 \rfloor) \quad (4)$$

Normal position sizing resumes when the account balance recovers to the pre-drawdown watermark.

4 Live Trading Deployment

The production system trades 0DTE options on both **SPXW** (S&P 500 Weeklys) and **QQQ** (Invesco Nasdaq-100 ETF), generating directional signals via the GBT ensemble and executing

strike selection and exit management via the RL agent. Discord alerts differentiate between these two model outputs, enabling real-time monitoring of both signal and execution layers.

4.1 System Architecture

The live deployment comprises three runtime components: a data feed process, a trading bot process, and the ThetaTerminal proxy. All three must be running simultaneously during market hours.

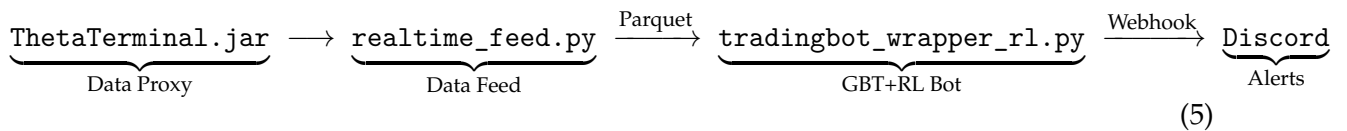


Figure 2: Live system data flow. All communication between feed and bot is via Parquet files in `rt_data/{YYYYMMDD}/`.

4.2 Environment Configuration

Before launching the system, configure the following environment variables in a `.env` file at the project root or via system environment:

Variable	Required	Description
DISCORD_WEBHOOK_URL	Yes	Discord webhook endpoint for the alerts channel
DISCORD_ROLE_PING	No	Role mention string (e.g., <code><@&ROLE_ID></code>) for @-pinging a role on trade alerts
THETADATA_DIR	No	ThetaData base directory (default: <code>D:\ThetaData</code>)
PROJECT_ROOT	Auto	Auto-detected from script location

Table 6: Environment variables for live deployment.

4.3 Step-by-Step Startup Sequence

4.3.1 Step 1: Start ThetaTerminal Proxy

The ThetaTerminal JAR file must be running before any data requests. It proxies all API calls to `http://127.0.0.1:25503/v3`:

```
java -jar ThetaTerminalv3.jar
```

Verify the terminal responds with no errors. An active ThetaData subscription with historical and live options access is required.

4.3.2 Step 2: Launch the Real-Time Data Feed

The feed service fetches options and spot data for both tickers every 60 seconds:

```
python services/realtime_feed.py --interval 60
```

Ticker	Options Symbol	0DTE Endpoints	Weekly Endpoints
SPX	SPXW	Greeks, OI, IV, OHLC	Greeks, OI
QQQ	QQQ	Greeks, OI, IV, OHLC	Greeks, OI

Table 7: Options data fetched per ticker per poll cycle.

What the feed downloads: In addition, the feed downloads 1-minute spot candles (underlying derived) for: **SPX, QQQ, VIX, and TLT**.

Startup behavior:

1. **Expiration resolution:** For each ticker (SPXW, QQQ), the feed queries ThetaData for available expirations and computes:
 - **0DTE:** Today's date (must be in the available expirations list).
 - **Weekly:** This week's Friday. If Friday is a market holiday, walk backward to Thursday, Wednesday, etc. If the weekly expiration coincides with the 0DTE expiration (e.g., today is Friday), the weekly is pushed to the following week's Friday and the walk-backward procedure is repeated.

Each ticker resolves its expirations independently, since SPXW and QQQ may have different available expiration dates.

2. **Historical backfill:** Downloads 5 previous trading days of spot data for SPX, QQQ, VIX, and TLT. This provides the historical Initial Balance (IB) levels required by features `dist_ib_high_D1...D5`.
3. **OHLC zero-fix:** At 09:30, the underlying derived data from ThetaData may contain zero values in the open, high, low, or close columns. The `_fix_ohlc_zeros` method handles this:
 - Rows where *all* OHLC columns are zero are dropped entirely.
 - Rows where *some* columns are zero: the zero values are replaced with the first non-zero value from the same row (e.g., if `open=0` but `close=5980`, then `open` is set to 5980).

Output file structure: All data is written as Parquet to `rt_data/{YYYYMMDD}/`:

```
rt_data/20260304/  
  SPXW_greeks_0dte_latest.parquet  
  SPXW_oi_0dte_latest.parquet  
  SPXW_iv_0dte_latest.parquet  
  SPXW_ohlc_0dte_latest.parquet  
  SPXW_greeks_weekly_latest.parquet  
  SPXW_oi_weekly_latest.parquet  
  QQQ_greeks_0dte_latest.parquet  
  QQQ_oi_0dte_latest.parquet  
  QQQ_iv_0dte_latest.parquet  
  QQQ_ohlc_0dte_latest.parquet  
  QQQ_greeks_weekly_latest.parquet  
  QQQ_oi_weekly_latest.parquet  
  spot_SPX_latest.parquet  
  spot_QQQ_latest.parquet  
  spot_VIX_latest.parquet  
  spot_TLT_latest.parquet
```

For a single-poll dry run (useful for validating data download without entering the main loop):

```
python services/realtime_feed.py --dry-run
```

4.3.3 Step 3: Launch the Trading Bot

```
python bots/tradingbot_wrapper_rl.py
```

The bot requires the following model files to be present:

- `models/trading_hybrid_wf.joblib` — GBT ensemble (directional classifier)
- `models/hybrid_normalizer_wf.npz` — Feature normalizer
- `rl_models/best_rl_agent.pt` — PPO agent (strike selection + exit management)

4.4 Dual-Ticker Processing Pipeline

The bot processes both SPX and QQQ in each polling cycle (every 30 seconds). For each ticker $\tau \in \{\text{SPX}, \text{QQQ}\}$:

1. **Data loading:** Reads the ticker-specific Parquet files from `rt_data/`. SPX reads `SPXW_*` files; QQQ reads `QQQ_*` files. A freshness check rejects any file older than 120 seconds.
2. **Greek exposure computation:** Merges Greeks and OI DataFrames, computes net exposures Γ_{net} , $\mathcal{V}_{\text{net}}$, Θ_{net} , etc. using the same `get_net_exposures_from_parquet` function as training.

3. **Feature engineering:** Builds a 163-dimensional feature vector $\mathbf{x}_t^{(\tau)} \in \mathbb{R}^{163}$, identical in structure to the training features. Per-ticker state (price history, IV history, IB levels, historical IB D-1...D-5) is tracked independently.

4. **GBT inference:**

$$(\hat{y}_t^{(\tau)}, \mathbf{p}_t^{(\tau)}) = \text{GBTEnsemble}(\text{normalize}(\mathbf{x}_t^{(\tau)})) \quad (6)$$

where $\hat{y}_t \in \{\text{SHORT}, \text{HOLD}, \text{LONG}\}$ and $\mathbf{p}_t = (p_{\text{short}}, p_{\text{hold}}, p_{\text{long}}) \in \Delta^2$ is the softmax probability vector.

5. **RL execution:** The directed signal is passed to the `IntegratedTradingSystem`, which uses the RL agent to select the optimal strike (delta bucket) from the live options chain, manage entry timing, and decide exit via the learned policy.
6. **Discord alerts:** Both GBT signals and RL executions generate tagged Discord messages (see §4.6).

4.5 Market Hours and Safety

Both the feed and the bot enforce Regular Trading Hours (09:30–16:00 ET):

- The feed polls only within 09:30–16:00 ET and skips weekends.
- The bot's `is_market_hours()` method returns `False` outside this window, causing the main loop to sleep for 60 seconds before rechecking.
- On startup before market open, the feed downloads historical backfill data; the bot loads historical IB levels from the backfilled files.

4.6 Discord Alert System

Trade signals, entries, exits, and status updates are transmitted to a Discord channel via HTTP POST to the configured `DISCORD_WEBHOOK_URL`. Alerts are tagged by model origin to distinguish between directional signals and execution events.

4.6.1 GBT Directional Signals — [GBM] Tag

When the GBT ensemble produces a non-HOLD directional prediction with confidence $c_t \geq c_{\min}$ (default $c_{\min} = 0.55$), a [GBM] alert is sent. These alerts are **deduplicated**: the same direction is not re-alerted until the GBT signal changes (e.g., `LONG` → `HOLD` → `LONG` generates two alerts, not a continuous stream).

```
[GBM] SPX LONG (67.2%)
SHORT: 12.3% | HOLD: 20.5% | LONG: 67.2%
RL: AGREES (LONG)
```

The alert includes:

- Ticker and predicted direction (LONG/SHORT).

- Full probability breakdown across all three classes.
- RL agreement status: whether the RL agent's current action aligns with the GBT direction.

4.6.2 RL Trade Execution — [RL] Tag

When the RL agent opens or closes a position, an [RL] alert is sent:

Trade open:

```
[RL] OPEN SPX LONG
Strike: 5980 | Delta: 0.35 | Bucket: otm_near
Entry: $4.20 | Confidence: 67.2%
```

Trade close:

```
[RL] CLOSE SPX LONG
Strike: 5980 | P&L: +12.3% ($62.50)
Hold: 23 min | Reason: agent_exit
```

4.6.3 Bot Status Messages

The bot sends [STATUS] messages on startup and shutdown:

```
[STATUS] Bot started
[STATUS] Bot stopped | Daily P&L: +$142.30
```

4.7 Expiration Resolution Logic

The weekly expiration resolution follows a deterministic algorithm designed to handle market holidays and coinciding expirations:

Definition 1 (Weekly Expiration Resolution). *Given today's date d and a set of available expirations $\mathcal{E}$:*

1. Let F_0 be the Friday of the current week: $F_0 = d + (4 - \text{weekday}(d)) \bmod 7$.
2. Walk backward from F_0 : for $k = 0, 1, 2, 3, 4$, check if $(F_0 - k) \in \mathcal{E}$ and $(F_0 - k) > d$.
3. If found and not equal to the 0DTE expiration, accept it as the weekly.
4. If no valid weekly is found (or found weekly = 0DTE): set $F_1 = F_0 + 7$ and repeat the walk-backward from F_1 .

This handles: (a) Good Friday and other exchange holidays where Friday expirations do not exist (falls back to Thursday); (b) a Wednesday trading day where the only remaining expiration this week is Thursday; (c) Friday itself, where the weekly must be next week to avoid collision with 0DTE.

4.8 Historical Data Requirements

The bot loads historical IB data for the previous 5 trading days for each ticker independently. If data files are missing (e.g., first-time deployment), the feed's `-dry-run` mode downloads them automatically:

```
# Download all historical data needed by the bot
python services/realtime_feed.py --dry-run
```

After running, verify that the following files exist for each of the last 5 trading days:

```
rt_data/YYYYMMDD/spot_SPX_latest.parquet
rt_data/YYYYMMDD/spot_QQQ_latest.parquet
rt_data/YYYYMMDD/spot_VIX_latest.parquet
rt_data/YYYYMMDD/spot_TLT_latest.parquet
```

5 Visualization and Diagnostics

5.1 Trade Chart

Plot all trades from a backtest CSV, colored by win/loss:

```
python neural/visualize_backtest_trades.py \
  --trades training_data/backtest_trades.csv \
  --ticker SPX \
  --save
```

5.2 Feature Importance

Generate SHAP-style feature importance from the GBT ensemble:

```
python neural/visualize_hybrid.py \
  --model models/trading_hybrid_wf.joblib \
  --normalizer models/hybrid_normalizer_wf.npz \
  --data training_data/training_data_derived.parquet \
  --model-size small \
  --ensemble \
  --save
```

5.3 Model Registry Inspection

The walk-forward window registry is saved to `models/window_registry.json` after each training run. It contains the rank, number of models, average profit factor, and rank score for every eligible window:

```
[
  {"window": 12, "n_models": 3, "pf_avg": 1.847,
   "rank_score": 1.532, "status": "PROD"},
  {"window": 11, "n_models": 2, "pf_avg": 1.612,
```

```

    "rank_score": 1.198, "status": "PROD"},
    ...
]

```

6 Operational Checklist

Step	Action	Verification
1	Start ThetaTerminal	API responds at <code>http://127.0.0.1:25503/v3</code>
2	Run data corrector	Zero-value anomalies cleared in <code>data_underlying_derived</code>
3	Collect training data	<code>training_data_derived.parquet</code> exists with $N > 100,000$ rows
4	Train GBT	<code>trading_hybrid_wf.joblib</code> updated; <code>window_registry.json</code> shows ≥ 5 PROD windows
5	Generate episode index	<code>episode_index.parquet</code> contains $N_{\text{ep}} > 5,000$ rows
6	Run RL preprocessing	<code>rl_options_cache_chunks/</code> contains one <code>.pkl</code> per trading day
7	Train PPO	<code>best_rl_agent.pt</code> saved; eval PF > 1.0
8	Backtest	Win rate $> 50\%$, profit factor > 1.2 on out-of-sample data
9	Start feed	<code>rt_data/</code> refreshing every 60 s
10	Start bot	First Discord alert received within 30 min of market open

Table 8: Pre-deployment operational checklist.

References

- [1] J. Schulman et al., “Proximal Policy Optimization Algorithms,” *arXiv:1707.06347*, 2017.
- [2] G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” *NeurIPS*, vol. 30, 2017.
- [3] ThetaData Systems, “ThetaData API v3 Documentation,” <https://docs.thetadata.us>, 2024.
- [4] L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why do tree-based models still outperform deep learning on typical tabular data?” *NeurIPS*, vol. 35, 2022.